

1. Introduction

These notes study how large language models can be extended beyond text to handle multiple modalities simultaneously. The central question is deceptively simple: if a language model already understands text at a deep level, how do we teach it to also see images, hear audio, or interpret video? The answer turns out to involve several non-trivial design choices, ranging from how to encode visual inputs to how to connect them to a pre-trained text backbone, and ultimately to whether the model should merely understand multi-modal inputs or also generate them.

We begin by reviewing the architecture and training pipeline of large language models themselves, since these serve as the backbone of all the multi-modal systems we study. We then examine the three-component design common to most multi-modal large language models: a vision encoder, a connector module, and a language model backbone. After establishing the architecture, we consider a harder problem: how to make the language model generate images, not just understand them. This leads us to visual autoregressive generation and the SEED and VAR architectures.

The second half of these notes shifts focus to efficiency. Large models are expensive to train and expensive to run at inference time. We study LoRA for parameter-efficient fine-tuning, KV cache for inference acceleration, and Mixture of Experts for combining large model capacity with manageable per-token computation. We conclude by examining how multi-modal foundation models are being applied to embodied intelligence and robotics, where a model must translate vision and language into physical actions.

2. Large Language Models: Architecture and Generation

Before we can understand multi-modal extensions, we need to understand what a large language model is and how it works. A large language model is a neural network trained to model the probability distribution over sequences of tokens. Given a vocabulary of size V and a sequence of tokens $w_1, w_2, \dots, w_T$, the model factorizes the joint probability autoregressively:

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1}).$$

At each step t , the model takes the preceding context as input and outputs a probability distribution over the entire vocabulary. Training minimizes the cross-entropy loss between the predicted distribution and the true next token.

Modern large language models use the Transformer decoder as their primary architecture. As discussed in previous lectures, the Transformer decoder applies causally masked multi-head self-attention, which allows each position to attend to all preceding positions but not to future positions. This causal masking ensures that the autoregressive factorization is respected: the prediction at position t cannot use information from positions $t + 1, t + 2, \dots$.

Text generation proceeds by sampling from the model iteratively. Given a prompt $w_1, \dots, w_s$, the model computes a distribution over the next token, samples w_{s+1} , appends it to the context, and repeats. This is the autoregressive or next-token prediction paradigm. It is simple, general, and has proven extraordinarily effective when applied at scale.

There are three main architectural variants. Encoder-only models such as BERT use bidirectional attention and are well-suited for understanding tasks. Decoder-only models such as GPT use causal attention and are designed for generation. Encoder-decoder models such as the original Transformer for machine translation process an input sequence with the encoder and generate an output sequence with the decoder using cross-attention. Modern large language models have largely converged on the decoder-only architecture for its simplicity and effectiveness in few-shot and zero-shot generalization.

3. Training Large Language Models

Training a large language model involves several stages that progressively specialize its capabilities.

The first stage is pre-training on a massive corpus of raw text. The objective is simply next-token prediction, and the data is typically drawn from web pages, books, code repositories, and scientific articles. Because no human annotation is required, it is feasible to collect data at the scale of hundreds of billions or even trillions of tokens. Pre-training gives the model broad world knowledge, language fluency, and basic reasoning capabilities.

The second stage is instruction fine-tuning, sometimes called supervised fine-tuning (SFT). Here, the model is trained on a curated dataset of prompt-response pairs that demonstrate helpful, accurate, and appropriately formatted behavior. This stage teaches the model to follow instructions and produce outputs that match human expectations.

The third stage involves alignment, typically using reinforcement learning from human feedback (RLHF) or a related method. A reward model is trained from human pairwise preferences and then used to further fine-tune the policy. This stage reduces harmful outputs, improves calibration, and aligns the model more closely with human values. The techniques underlying this stage were studied in detail in the previous set of notes on deep reinforcement learning.

4. From LLMs to Multi-modal Large Language Models

A language model trained only on text cannot process an image. It has no mechanism to ingest pixel values or extract visual features. Multi-modal large language models (MLLMs) extend the language model by adding components that translate non-text modalities into a form the language model can process.

The primary motivation is that many real-world tasks involve reasoning over multiple modalities simultaneously. Describing an image, answering a question about a video, transcribing speech, or controlling a robot all require connecting visual or auditory inputs with language understanding. A model that can handle multiple modalities is not simply more capable at individual tasks; it can also draw on cross-modal synergies, for example using language to guide visual attention or using visual context to disambiguate language.

The capabilities of modern MLLMs are remarkable. They can describe complex scenes, reason about spatial relationships, read and interpret text within images, solve visual mathematics problems, and generate coherent narratives grounded in visual content. These capabilities emerge from the combination of powerful vision encoders, effective connector modules, and very large language model backbones trained at scale.

5. The Three-Component Architecture of MLLMs

Most multi-modal large language models share a common three-component architecture.

The first component is a vision encoder. Its role is to convert an image or video frame into a sequence of feature vectors that summarize the visual content. Common choices include ViT-based encoders pre-trained with contrastive objectives such as CLIP, as well as convolutional encoders. A CLIP-trained ViT is particularly popular because it has already learned a vision representation that is semantically aligned with text: images and their corresponding captions are mapped to nearby points in feature space.

The second component is a connector module, also called the multimodal alignment module. Its role is to bridge the gap between the vision encoder’s output space and the language model’s input space. The vision encoder produces a sequence of visual feature vectors, but the language model expects a sequence of token embeddings from its own vocabulary. The connector must translate between these two representations. Different connector designs represent different trade-offs between expressiveness and simplicity, as we will examine in the next section.

The third component is the language model backbone. This is typically a large, pre-trained decoder-only Transformer. It receives a mixed sequence of visual tokens (produced by the connector) and text tokens, and processes them jointly using causal self-attention. The language model’s extensive world knowledge and reasoning capabilities are leveraged to interpret and respond to the visual inputs.

For tasks that require generating non-text outputs such as images or audio, a fourth component is sometimes added: a generator or decoder that takes the language model’s output and converts it back into the target modality.

6. Connecting Vision and Language: Learnable Connectors

The connector is the most critical and variable component in MLLM design. It must align two quite different feature spaces while preserving as much relevant information as possible.

The simplest connector is a multilayer perceptron (MLP) applied to each visual feature vector independently. LLaVA, for example, uses a two-layer MLP to project each ViT patch feature directly into the language model’s embedding space. The resulting sequence of projected vectors is prepended to the text token sequence and processed by the language model. This approach is extremely simple and computationally cheap, yet it works well in practice when both the vision encoder and language model are powerful and well-aligned through pre-training.

A more sophisticated connector is the Q-Former, used in BLIP-2. Instead of passing all patch features to the language model, Q-Former maintains a small set of learnable query vectors (typically 32 queries) that attend to the visual features through cross-attention. These queries distill the visual content into a fixed-length sequence of vectors, compressing the potentially long sequence of patch features. The queries are designed to extract information relevant to language tasks by training with an image-text matching objective. The compression is advantageous because it reduces the number of visual tokens that the language model must process, reducing computational cost.

A third family of connectors inserts cross-attention layers between the language model’s own layers. Flamingo, for example, interleaves frozen language model layers with learned cross-attention modules that attend directly to the visual features at every depth. This approach allows visual information to influence the language model at multiple levels of abstraction, potentially enabling richer integration of visual and linguistic content.

The choice of connector also affects how visual information is fused with text. Token-level fusion concatenates visual and text tokens into a single sequence, allowing the language model’s self-attention to mix them freely. Feature-level fusion, by contrast, uses dedicated cross-attention to inject visual information into the processing stream without directly mixing it with text tokens. Each approach has different inductive biases about how vision and language should interact.

7. Generating Images with Language Models: Visual Autoregressive Approaches

So far we have described MLLMs that understand multi-modal inputs and generate text. A more ambitious goal is to have the same model generate images as outputs. This requires the model to produce pixel-level or feature-level image content, which is far more demanding than predicting discrete text tokens.

One natural approach is visual autoregressive generation. The key observation is that if images can be represented as sequences of discrete visual tokens, then image generation becomes a next-token prediction problem, identical in form to text generation. The language model already knows how to predict the next token; we just need to give it a vocabulary of visual tokens in addition to text tokens.

The main technical challenge is constructing this visual vocabulary. Continuous image features cannot be directly fed to a language model that operates over a discrete vocabulary. We need a way to discretize image features into a finite codebook, so that each image can be represented as a sequence of integer indices. Vector Quantization (VQ-VAE or VQ-GAN) achieves this by training an encoder that maps image patches to discrete codes and a decoder that reconstructs the image from those codes. The resulting codebook is the visual vocabulary.

8. SEED and SEED-X: Causal Semantic Tokenization

SEED (from Tencent) is a comprehensive visual autoregressive system that addresses both understanding and generation within a single framework. Its design involves five carefully orchestrated components.

First, a pre-trained ViT encoder (initialized from BLIP-2) extracts two-dimensional feature maps from the input image. Second, a causal Q-Former transforms these features into a one-dimensional sequence with a causal dependency structure, so that each visual feature can only attend to spatially preceding features. This causality is essential for autoregressive generation: the model must predict visual tokens one at a time, each conditioned on earlier tokens.

Third, the sequence of continuous features from the Q-Former is discretized using a VQ codebook, yielding a sequence of discrete visual tokens. Fourth, during generation, the language model outputs a sequence of discrete visual token indices. A Reverse Q-Former then maps these tokens back into a continuous feature space compatible with the decoder. Fifth, the continuous features are passed to a pre-trained Stable Diffusion UNet decoder, which generates the final image pixels.

The genius of this design is that the language model operates entirely in the space of discrete tokens, whether text or visual. It can freely interleave text and image tokens in a single autoregressive sequence, enabling tasks like image captioning, image-guided text generation, and text-to-image generation within a unified model.

SEED-X extends this framework with a more flexible visual representation scheme that supports dynamic image resolutions. The training strategy employs two objectives applied jointly:

causal Q-Former features are trained with an image-text contrastive objective to ensure semantic content, while the VQ codebook and Reverse Q-Former are trained with a dual reconstruction objective that enforces both semantic fidelity and fine-grained detail recovery. The inference pipeline maps language model outputs directly to visual features, which are then passed to a visual de-tokenizer for high-quality image generation.

9. Visual Scale Autoregressive: VAR

The SEED framework flattens an image into a one-dimensional sequence of tokens and predicts them left to right, just as language models process text. VAR (Visual Autoregressive modeling, from ByteDance) proposes a different inductive bias: instead of predicting the next token, the model predicts the next scale.

The motivating observation is that images have a natural hierarchical structure. A low-resolution version of an image captures the global layout and main semantic content. Successively higher-resolution versions add finer details. A multi-scale representation therefore encodes an image as a sequence of token maps at increasing resolutions: $r_1, r_2, \dots, r_K$ where r_k is a token map at resolution k .

Training VAR requires two stages. In the first stage, a multi-scale VQ-VAE is trained to encode each image into a hierarchy of discrete token maps. Each resolution level is encoded independently using a quantizer, yielding a sequence of token maps of sizes $1 \times 1, 2 \times 2, 4 \times 4, \dots, K \times K$ (or similar). The decoder is trained to reconstruct the image from all resolution levels jointly.

In the second stage, a VAR Transformer is trained autoregressively over the sequence of token maps. Given the token maps at all resolutions up to level $k - 1$, the model predicts the entire token map at resolution k as a single step. This is next-scale prediction rather than next-token prediction. Formally, the model predicts the conditional distribution

$$P(r_k \mid r_1, r_2, \dots, r_{k-1}),$$

where each r_k is an entire token map. Within each token map, tokens are predicted in raster-scan order with a standard next-token prediction loss.

This design has several advantages. By predicting coarse structure before fine detail, the model can maintain global coherence. The receptive field of each prediction step covers the full set of coarser token maps, so every fine-grained token is generated with full awareness of the global content. Empirically, VAR achieves strong performance on image generation benchmarks and scales well with model size, following scaling laws similar to those observed in language modeling.

10. Efficient Training with LoRA

As language models and multi-modal models grow in size, fine-tuning all parameters on downstream tasks becomes increasingly expensive. A model with tens or hundreds of billions of parameters requires enormous memory and computational resources to train, even for a single epoch. Parameter-efficient fine-tuning methods address this by modifying only a small subset of parameters.

LoRA, which stands for Low-Rank Adaptation, is the most widely used such method. Its core observation is that the update to a weight matrix during fine-tuning tends to have low intrinsic rank. If the pre-trained weight matrix for a layer is $W_0 \in \mathbb{R}^{d \times k}$, then instead of learning a dense update $\Delta W \in \mathbb{R}^{d \times k}$, LoRA parameterizes the update as a product of two low-rank matrices:

$$\Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k},$$

where $r \ll \min(d, k)$ is the rank hyperparameter. The adapted weight is then $W = W_0 + \Delta W = W_0 + BA$. During fine-tuning, W_0 is frozen and only B and A are trained. The number of trainable parameters for this layer is $r(d + k)$ instead of dk . For typical values $r = 8$ and $d = k = 4096$, this is a reduction by a factor of more than 500.

An important initialization detail is that B is initialized to zero, while A is initialized randomly. Because $B = 0$ initially, $\Delta W = BA = 0$ at the start of training. This means the adapted model begins exactly at the pre-trained model, and fine-tuning adjusts it from this stable starting point. If instead ΔW were initialized randomly, the model would begin in a different configuration from the pre-trained model, potentially disrupting the knowledge encoded by pre-training.

LoRA is typically applied to the attention weight matrices W_Q , W_K , W_V , and W_O of each Transformer layer. The feedforward layers may also be adapted but are sometimes left frozen to save parameters. In the multi-modal setting, LoRA is often used to fine-tune the language model backbone on task-specific data while keeping the vision encoder fixed, since the vision encoder is typically not the bottleneck.

Several variants of LoRA have been proposed to further improve its effectiveness. Some variants adapt the rank dynamically across layers, allocating more capacity to layers where it is most needed. Others extend the low-rank decomposition to the biases, layer norm parameters, or embedding matrices. The common thread in all these variants is the idea that the fine-tuning update lives in a low-dimensional subspace of the full parameter space.

11. Efficient Inference with KV Cache

Even after fine-tuning, running a large language model at inference time is expensive. In autoregressive generation, the model processes the entire context at each step to predict the next token. Naively, this requires computing key and value matrices for all tokens in the context at every step, even for tokens that were processed in previous steps and have not changed.

The KV cache is a simple but extremely effective optimization that avoids this redundant computation. At each step, the key and value matrices for all previously processed tokens are stored in a cache. When processing the new token at position t , the model only needs to compute the key and value for position t . The attention mechanism then uses the cached keys and values from positions $1, \dots, t - 1$ together with the new key and value at position t to compute the attention output.

To see why this is correct, recall that in causal attention the output at position t is

$$\text{out}_t = \sum_{j=1}^t \alpha_{t,j} V_j, \quad \alpha_{t,j} \propto \exp\left(\frac{Q_t \cdot K_j}{\sqrt{d_k}}\right),$$

where Q_t is the query at the new position and K_j, V_j are the keys and values at position j . Since K_j and V_j depend only on the input at position j and the model parameters (which are fixed), they are the same every time position j is processed. The KV cache simply stores them after their first computation and reuses them in all subsequent steps.

The memory cost of the KV cache scales as $O(N \cdot L \cdot d)$ where N is the sequence length, L is the number of Transformer layers, and d is the head dimension. For very long sequences or large models, this can become substantial. Several techniques have been proposed to reduce KV cache memory, including grouped-query attention (which shares keys and values across multiple query heads), multi-query attention, and various quantization and pruning strategies. Managing KV cache memory is one of the central practical challenges in deploying large language models at scale.

12. Mixture of Experts

A standard feedforward network in a Transformer applies the same computation to every token. All parameters are used for every forward pass. Mixture of Experts (MoE) relaxes this constraint by maintaining multiple expert networks and routing each token to only a subset of them.

In a standard MoE feedforward layer, there are E expert networks $\{f_1, f_2, \dots, f_E\}$, each a small MLP. A gating network takes the input token x and produces a routing distribution over experts. In sparse MoE, only the top- k experts (typically $k = 1$ or $k = 2$) receive each token. If $g_i(x)$ denotes the gate value for expert i , the output is

$$\text{MoE}(x) = \sum_{i \in \text{top-}k} g_i(x) \cdot f_i(x).$$

The key insight is that the total number of parameters scales with E , but the computation per token scales only with k . By choosing E large and k small, one can have a model with many more parameters than would be feasible to activate for every token. The hypothesis is that a model with more total parameters but sparse activation can achieve better performance than a dense model of the same computational budget.

This creates a favorable trade-off: large total capacity for encoding world knowledge and handling diverse inputs, but low per-token inference cost. The challenge lies in training the routing network to use all experts effectively. If the gating consistently routes tokens to only a few popular experts, the other experts remain undertrained and contribute little. Load balancing losses are typically added to the training objective to encourage uniform utilization of experts.

MoE has been applied both to decoder-only language models and to multi-modal architectures. MoE-LLaVA, for example, replaces the feedforward layers in the language model with MoE layers, allowing different experts to specialize in different types of content while keeping inference cost manageable. This is particularly relevant in the multi-modal setting, where different experts might naturally specialize in visual versus textual reasoning.

A recent line of work on dense training, sparse inference explores whether models can be trained densely (activating all experts for all tokens during training, which improves gradient flow) and then converted to sparse inference (activating only top- k experts per token at test time) without a significant performance drop. This approach attempts to decouple the training and inference benefits of MoE architectures.

13. Multi-modal Foundation Models for Robotics

One of the most exciting application domains for multi-modal foundation models is robotics. A robot must perceive its environment (vision), understand instructions (language), plan sequences of actions (reasoning), and execute those actions (control). This hierarchy maps naturally onto the capabilities of MLLMs, and recent work has explored using such models to drive robotic behavior at multiple levels of abstraction.

At the highest level, a vision-language model can perform long-horizon planning: given a visual observation of the environment and a natural language task description, it generates a plan expressed as a sequence of high-level sub-goals. PaLM-E, for example, is a very large multi-modal model (over 500 billion parameters) that processes both images and text in a unified sequence and generates natural language plans for embodied tasks. It demonstrates that a single model trained on diverse multi-modal data can plan across a wide range of settings.

At an intermediate level, models like RoboPoint take a different approach. Instead of generating free-form text plans, RoboPoint outputs spatial affordance predictions: given an image and a task description, it predicts a set of keypoints in the image indicating where the robot should interact. This structured output is easier to consume by downstream motion planners than open-ended text.

ReKep takes an even more structured approach, using a VLM to generate relational keypoint constraints between objects in the scene. These constraints specify geometric relationships that the robot must satisfy to accomplish the task (for example, “the cup must be upright” or “the lid must be above the container”). The VLM’s semantic understanding is used to specify what needs to be achieved, while a separate trajectory optimizer finds the actual motion plan that satisfies the constraints.

14. From VLM to VLA: Vision-Language-Action Models

Vision-Language-Action models (VLAs) take the most direct approach: they train a model that directly maps visual observations and language instructions to robot actions, without any separate planning or optimization step.

OpenVLA, developed at Stanford and Google, is an open-source VLA based on a 7-billion-parameter language model fine-tuned on a large robot demonstration dataset. The input to OpenVLA consists of a camera image (processed by a ViT-based vision encoder) and a natural language instruction. The output is a sequence of action tokens, where each token represents a discretized joint position or end-effector movement. By framing action prediction as token generation, OpenVLA leverages the full machinery of the language model, including its ability to reason about language, to produce context-appropriate actions.

Training OpenVLA involves two stages. In the first stage, the model is pre-trained on a large-scale robot dataset (14 days on 64 GPUs), covering diverse tasks and robot embodiments. In the second stage, the model is fine-tuned on data from the specific target task and robot (1 day on a single GPU). This two-stage procedure follows the same pre-training plus fine-tuning paradigm established in NLP: broad pre-training instills general capabilities, and task-specific fine-tuning adapts them to the target distribution.

More recent models such as $\pi_{0.5}$ and $\pi_{0.6}$ from Physical Intelligence push this paradigm further. $\pi_{0.5}$ emphasizes open-world generalization: the ability to handle novel tasks and environments not seen during training. This requires the model to generalize from its pre-trained knowledge rather than memorize specific task demonstrations. $\pi_{0.6}$ extends this by enabling the model to improve from its own experience: after deployment, the robot collects new data from its interactions and uses it to further fine-tune the model. This closed-loop learning is a step toward truly autonomous robotic agents that continuously improve through interaction with the world.

15. Concluding Remarks

The unifying theme of these notes is that a powerful language model backbone, combined with appropriate adaptation mechanisms, can serve as the foundation for multi-modal understanding and generation. The Transformer’s general-purpose attention mechanism, originally designed for text, has proven surprisingly effective when extended to images, video, and actions. What changes across applications is primarily how non-text modalities are encoded and how they are connected to the language backbone.

The efficiency techniques studied in the second part of these notes are not peripheral engineering concerns. They are central to what makes large multi-modal models practical. Without LoRA, fine-tuning a multi-modal model on task-specific data would require hardware accessible only to the largest organizations. Without KV cache, autoregressive inference with a long context would be far too slow for interactive applications. Without MoE, the capacity needed for multi-modal reasoning would be prohibitively expensive to activate for every token.

Looking across the systems discussed, from BERT to SEED to VAR to VLA, one can identify a recurring pattern: start with a strong pre-trained backbone, define a task-appropriate tokenization or discretization of the new modality, connect modalities with a learned alignment module, and fine-tune the whole system end-to-end with appropriate objectives. Understanding this pattern allows one to reason about new architectures and applications even before they are published, by asking which component serves each role and how the components interact.